

Netflix-Style TV UI — Portfolio Project Implementation Plan

What We're Building

A TV-style content browser that demonstrates mastery of the exact engineering challenges Netflix TVUI engineers face daily. This is NOT a web app that looks like a TV — it's built to behave like one: keyboard-only navigation, no mouse, virtualized rendering for constrained devices, custom focus management, smooth animations, and performance instrumentation.

This project is your interview artifact for Netflix TVUI Engineering.

Stack: React 18 + TypeScript + Vite (no Next.js — keep it simple like a TV app) **No external UI libraries.** No Tailwind, no styled-components, no Material UI. TV apps don't use these. Build your own styling system — this demonstrates understanding of Netflix's approach. **Canvas-style rendering optional.** ReactDOM is fine for the portfolio piece, but architect it so the rendering layer is abstracted and could be swapped for a custom renderer. This shows you understand the Gibbon concept.

Project Structure

```
tv-ui/
├─ src/
│   ├─ main.tsx           # Entry point
│   ├─ App.tsx            # Root app component
│   ├─ engine/
│   │   ├─ FocusEngine.ts # Custom focus management system
│   │   ├─ InputManager.ts # Keyboard/remote input handler
│   │   ├─ ScrollEngine.ts # Smooth row scrolling logic
│   │   └─ VirtualList.ts  # Virtualization engine
│   └─ components/
│       ├─ Shell.tsx       # App shell with header + content area
│       ├─ ContentRow.tsx  # Horizontal scrollable content row
│       ├─ ContentTile.tsx # Individual content card (poster + metadata)
│       ├─ HeroBanner.tsx  # Large featured content banner at top
│       ├─ FocusRing.tsx   # Visual focus indicator component
│       ├─ RowTitle.tsx    # Row label ("Trending Now", "Continue Watching")
│       └─ DetailOverlay.tsx # Expanded detail view when pressing Enter on a
tile
│   │   └─ PerformanceHUD.tsx # FPS counter, memory, render time overlay
│   └─ hooks/
│       └─ useFocus.ts      # Hook for components to participate in focus
```

```

system
├── useVirtualization.ts      # Hook for virtualized row rendering
├── useAnimation.ts          # Hook for smooth scroll animations
├── usePerformance.ts        # Hook for performance measurement
├── useInputNavigation.ts    # Hook for keyboard navigation
├── state/
│   ├── store.ts            # Redux store setup
│   ├── slices/
│   │   ├── focusSlice.ts   # Focus state (which row, which tile)
│   │   ├── contentSlice.ts  # Content data (rows, tiles, metadata)
│   │   ├── uiSlice.ts      # UI state (detail overlay open, scroll positions)
│   │   └── perfSlice.ts    # Performance metrics state
│   └── selectors.ts        # Memoized selectors
├── data/
│   ├── mockContent.ts      # Mock content data (movie titles, posters,
metadata)
│   └── rowConfigs.ts        # Row configurations (type, content source,
display rules)
├── styles/
│   ├── theme.ts            # Color palette, spacing, typography tokens
│   └── styleEngine.ts       # Custom CSS-in-JS system (simplified Gibbon-like
approach)
├── animations.ts            # Animation timing functions and presets
├── utils/
│   ├── performance.ts      # Performance measurement utilities
│   ├── debounce.ts         # Input debouncing for remote control
│   └── constants.ts        # Grid dimensions, timing constants
├── public/
│   └── index.html
├── package.json
├── tsconfig.json
├── vite.config.ts
└── README.md

```

Phase 1: Focus Engine + Keyboard Navigation (Day 1)

Goal

Build the core interaction model: a 2D grid of items navigable entirely by keyboard. This is the foundation everything else builds on.

Step 1: Project Setup

```
bash

npm create vite@latest tv-ui -- --template react-ts
cd tv-ui
npm install @reduxjs/toolkit react-redux
npm install -D @types/react @types/react-dom
npm run dev
```

No other dependencies. Everything else we build from scratch.

Step 2: Focus Engine (`src/engine/FocusEngine.ts`)

The focus engine is the brain of a TV UI. It tracks a 2D cursor position (row index, tile index) and handles directional navigation.

```
typescript

// Core concept: the focus engine manages a 2D grid of focusable items
// It knows: which row has focus, which tile within that row has focus
// It handles: arrow key navigation, edge cases, focus wrapping

interface FocusState {
  rowIndex: number;
  tileIndex: number;
  // Track the "remembered" tile index per row so when you go up/down,
  // you return to roughly the same horizontal position
  rowMemory: Map<number, number>;
}

interface FocusableGrid {
  rows: { id: string; tileCount: number }[];
}
```

Key behaviors to implement:

- **Left/Right:** Move within the current row. Stop at edges (don't wrap).
- **Up/Down:** Move between rows. Remember the tile index from the previous row so you return to a similar position (Netflix does this — if you're on tile 5 in row 2 and press Down to row 3 which has 3 tiles, focus goes to tile 3. When you go back Up, it returns to tile 5).
- **Enter:** Select the focused tile (open detail overlay).
- **Back/Escape:** Close the detail overlay.

- **Debounce rapid key presses:** If someone holds down the arrow key, don't fire 60 events per second. Debounce to ~10 per second for smooth scrolling.

Step 3: Input Manager (`src/engine/InputManager.ts`)

```
typescript

// Maps keyboard events to abstract navigation actions
// This abstraction means the same engine works for keyboards,
// remote controls, and gamepads

type NavigationAction = 'UP' | 'DOWN' | 'LEFT' | 'RIGHT' | 'SELECT' | 'BACK';

// Key mappings:
// Arrow keys → directional navigation
// Enter/Space → SELECT
// Escape/Backspace → BACK
// WASD → also directional (for developer convenience)

// Implement key repeat handling:
// - First press: immediate action
// - Hold: wait 300ms, then repeat at ~100ms intervals
// - This prevents jank from OS-level key repeat rates
```

Step 4: Wire Focus to React

Create `useFocus` hook that components use to:

- Register themselves as focusable
- Know if they currently have focus
- Receive focus/blur callbacks for animation triggers

Create `useInputNavigation` hook that:

- Listens for keyboard events on the window
- Passes them through InputManager
- Dispatches focus changes through Redux

Step 5: Build Basic Grid Layout

Create `Shell.tsx`, `ContentRow.tsx`, and `ContentTile.tsx`:

- Shell renders 6-8 rows vertically

- Each row renders 10-15 tiles horizontally
- Only the focused tile has a visual indicator (scale up + border glow)
- Use mock data: movie titles and placeholder poster images

No scrolling yet. Just render the grid and make arrow keys move focus around.

Verification

- Arrow keys navigate focus through a 2D grid
 - Focus wraps correctly at row edges
 - Up/Down remembers horizontal position
 - Enter opens a detail overlay, Escape closes it
 - No mouse interactions work (by design)
-

Phase 2: Virtualized Rendering (Day 2)

Goal

Only render what's visible. On a real TV with 512MB RAM, you can't have 200 tile components mounted simultaneously.

Step 1: Virtual Row List (`src/engine/VirtualList.ts`)

typescript

```
// Only mount rows that are visible in the viewport + 1 row above and below
// (the buffer row enables smooth scroll animation before the next row mounts)
//
// With 8 rows visible and 1 buffer each direction, you mount 10 rows max
// Even if you have 30 rows of content, only 10 React components exist

interface VirtualListConfig {
  totalItems: number;
  visibleCount: number;
  bufferCount: number; // typically 1-2
  itemHeight: number;
  scrollPosition: number;
}

// Returns: { startIndex, endIndex, offset }
// startIndex/endIndex = which rows to mount
// offset = CSS translateY to position them correctly
```

Step 2: Virtual Tile List (within each row)

```
typescript

// Same concept but horizontal within each row
// Only mount tiles visible in the row viewport + 1 buffer on each side
//
// A row shows ~6 tiles. With 1 buffer each side, mount 8 tile components
// Even if the row has 50 items, only 8 React components exist

// This is critical for Netflix: their "Continue Watching" row could have
// 100+ items, but only ~8 are ever rendered
```

Step 3: Implement `useVirtualization` hook

```
typescript

// This hook takes: total item count, visible count, current scroll position
// Returns: items to render, their positions, and a scroll offset
// Components use this to know which children to mount
```

Step 4: Apply to ContentRow and Shell

Modify `Shell.tsx` to use vertical virtualization (only mount visible rows). Modify `ContentRow.tsx` to use horizontal virtualization (only mount visible tiles).

Step 5: Measure the difference

Add a component counter that shows how many React components are currently mounted.

Before virtualization: all rows $\times$ all tiles (e.g., $30 \times 15 = 450$ components).

After virtualization: ~ 10 rows $\times$ ~ 8 tiles = ~ 80 components.

Display this in the Performance HUD.

Verification

- Only visible rows and tiles are mounted (check React DevTools)
 - Scrolling to new rows triggers mount/unmount correctly
 - No visual gaps or pop-in when scrolling
 - Memory usage stays flat regardless of total content count
-

Phase 3: Smooth Scroll Animations (Day 3)

Goal

When focus moves to a tile at the edge of the visible area, the row (or entire page) smoothly slides to reveal more content. This must be 60fps on any device.

Step 1: Scroll Engine (`src/engine/ScrollEngine.ts`)

```
typescript
```

```
// When focus moves Right and approaches the edge of visible tiles,
// the row scroll position updates to reveal the next tiles
//
// Animation approach: use requestAnimationFrame for smooth interpolation
// NOT CSS transitions (they're harder to coordinate with virtualization)
//
// Easing function: cubic-bezier ease-out (fast start, gentle stop)
// Duration: 200-300ms for tile-to-tile, 400ms for row-to-row

interface ScrollAnimation {
  from: number;
  to: number;
  startTime: number;
  duration: number;
  easing: (t: number) => number;
}

// Implement spring physics (optional but Netflix-quality):
// Instead of fixed duration easing, use a spring model
// spring(stiffness: 300, damping: 25)
// This gives a natural "settle" feel that fixed easing can't match
```

Step 2: Row Scroll Behavior

When focus is on tile 5 and the user presses Right:

1. Focus moves to tile 6 (instant)
2. If tile 6 is near the right edge, start scroll animation
3. Row slides left to center the focused tile (or keep it ~2 tiles from the right edge)
4. Virtualization mounts new tiles as they scroll into view
5. Old tiles unmount as they scroll out of view

Netflix's specific behavior: the focused tile is usually the 2nd or 3rd from the left edge, not centered. This means you always see what's ahead of your current position.

Step 3: Vertical Scroll Behavior

When focus moves Down to a new row:

1. Focus moves to the next row (instant)
2. Page smoothly scrolls to keep the focused row in the viewport
3. The focused row should be near the top (not centered) — this is Netflix's pattern

4. Hero banner at the top shrinks as you scroll down (parallax effect)

Step 4: Integrate with Virtualization

The scroll engine and virtual list must work together:

- Scroll position changes → virtual list recalculates which items to render
- New items mount at the correct position → no visual pop-in
- Unmounted items happen off-screen → no visual disappearance

Verification

- Scrolling is smooth 60fps (use Chrome DevTools Performance tab)
 - No jank when new tiles mount during scroll
 - Focus always stays visible on screen
 - Hero banner parallax works on vertical scroll
 - Scroll settles naturally (easing or spring)
-

Phase 4: Styling System + Visual Polish (Day 4)

Goal

Build a custom styling system (no CSS frameworks) and make the app look like a premium TV experience.

Step 1: Style Engine (`src/styles/styleEngine.ts`)

typescript

```
// Netflix's Gibbon doesn't use CSS. Build a simplified version of this concept:
// - Styles are JavaScript objects, not CSS strings
// - Properties map to GPU-friendly rendering (transforms, opacity, background)
// - No CSS cascade – every component's styles are explicit and complete
// - This is similar to React Native's StyleSheet

// Example:
const tileStyles = {
  default: {
    width: 230,
    height: 130,
    borderRadius: 4,
    backgroundColor: '#1a1a1a',
    transform: 'scale(1)',
    transition: 'transform 200ms ease-out',
    opacity: 0.7,
  },
  focused: {
    transform: 'scale(1.15)',
    opacity: 1,
    boxShadow: '0 8px 24px rgba(0,0,0,0.6)',
    zIndex: 10,
    border: '3px solid #e50914', // Netflix red
  },
};
```

Step 2: Theme ((src/styles/theme.ts))

typescript

```

// Netflix TV aesthetic
const theme = {
  colors: {
    background: '#141414',
    surface: '#1a1a1a',
    surfaceHover: '#2a2a2a',
    focusRing: '#e50914',      // Netflix red
    text: '#ffffff',
    textSecondary: '#999999',
    textMuted: '#666666',
    gradient: 'linear-gradient(180deg, transparent 0%, #141414 100%)',
  },
  spacing: {
    rowGap: 48,                // vertical gap between rows
    tileGap: 8,                // horizontal gap between tiles
    edgePadding: 56,           // left/right screen padding
    headerHeight: 68,
  },
  typography: {
    rowTitle: { fontSize: 18, fontWeight: 600, letterSpacing: 0.5 },
    tileTitle: { fontSize: 13, fontWeight: 500 },
    tileMetadata: { fontSize: 11, fontWeight: 400 },
    heroTitle: { fontSize: 48, fontWeight: 700 },
    heroSynopsis: { fontSize: 16, fontWeight: 400, lineHeight: 1.4 },
  },
  animation: {
    focusScale: 1.15,
    unfocusScale: 1,
    focusDuration: 200,        // ms
    scrollDuration: 300,        // ms
    rowScrollDuration: 400,    // ms
  },
  tile: {
    width: 230,
    height: 130,
    borderRadius: 4,
  },
};

```

Step 3: Visual Components

ContentTile polish:

- Poster image (use placeholder images — `picsum.photos` or TMDB API for real posters)
- Title text below the poster (only visible when row is focused)
- Focus state: scale up, add Netflix-red border, increase opacity, drop shadow
- Unfocused tiles in focused row: slight opacity boost vs tiles in unfocused rows

HeroBanner:

- Large background image (fill the top 40% of screen)
- Title text over a gradient overlay
- Synopsis text (2-3 lines)
- "Play" and "More Info" buttons (focused state shows highlight)
- Parallax: banner shrinks and fades as you scroll down into rows

RowTitle:

- "Trending Now", "Continue Watching", "New Releases", etc.
- Fades in when the row approaches the focused area

DetailOverlay:

- When pressing Enter on a tile: overlay slides up from bottom
- Shows: large poster, title, synopsis, metadata (year, rating, genre)
- "Play", "Add to List", "Like" buttons navigable with Left/Right
- Escape dismisses

Step 4: Font

Use Netflix's actual font family approach:

- Netflix Sans (proprietary) → use Helvetica Neue or system-ui as fallback
- Weights: 400 (body), 500 (metadata), 600 (titles), 700 (hero)
- Don't use Google Fonts — TV apps use system fonts for performance

Verification

- App looks like a premium TV streaming interface
- Focus animations are smooth and feel responsive
- Visual hierarchy is clear (focused vs unfocused content)
- Hero banner provides visual impact

- Detail overlay works and dismisses cleanly
 - No external CSS frameworks used — all custom
-

Phase 5: Performance HUD + Metrics (Day 5)

Goal

Instrument the app with performance metrics and display them in a toggleable HUD. This demonstrates that you think about performance as a first-class concern.

Step 1: Performance Measurement (`src/utils/performance.ts`)

typescript

```
// Measure these metrics continuously:

// 1. FPS (frames per second)
//   Use requestAnimationFrame loop to measure frame timing
//   Display: current FPS + rolling average

// 2. Mounted component count
//   Track how many React components are mounted
//   Shows virtualization is working

// 3. Render time per frame
//   Use Performance API to measure how long each render takes
//   Highlight when renders exceed 16ms (60fps budget)

// 4. Memory usage
//   Use performance.memory API (Chrome only)
//   Track heap size over time

// 5. Input latency
//   Measure time from keydown event to visual focus change
//   Should be < 50ms for responsive feel

// 6. Scroll smoothness
//   Track dropped frames during scroll animations
//   Flag if any frame exceeds 16ms
```

Step 2: Performance HUD Component

```
typescript

// Toggleable overlay (press 'P' to toggle)
// Shows all metrics in a compact, monospace display
// Semi-transparent background, positioned in top-right corner
// Updates every 500ms (don't update every frame – that would impact perf!)
//
// Layout:
// ┌
// │ FPS: 60 (avg: 59.8) │
// │ Components: 84      │
// │ Render: 4.2ms       │
// │ Memory: 42MB        │
// │ Input lag: 12ms     │
// │ Dropped: 0 frames   │
// └
```

Step 3: Performance Annotations

Add markers to the Performance API timeline so Chrome DevTools shows labeled sections:

- `performance.mark('focus-change-start')` → `performance.mark('focus-change-end')`
- `performance.mark('scroll-animation-start')` → `performance.mark('scroll-animation-end')`
- `performance.mark('row-mount')` → `performance.mark('row-mount-end')`

These show up in Chrome DevTools Performance tab and demonstrate that you think about observability.

Verification

- HUD displays accurate metrics
- FPS stays at 60 during all interactions
- Component count stays bounded (virtualization working)
- Render time stays under 16ms
- Input latency is under 50ms
- Performance markers appear in DevTools

Phase 6: Final Polish + Documentation (Day 6-7)

Step 1: Edge Case Handling

Test and fix these specific edge cases:

- First tile in first row: Left and Up should do nothing (no crash)
- Last tile in last row: Right and Down should do nothing
- Row with 3 tiles → Down to row with 15 tiles → Up: focus returns to tile 3
- Rapid key presses (hold arrow key): smooth scrolling, no jank
- Detail overlay open: arrow keys navigate overlay buttons, not the grid behind it
- Window resize: recalculate visible tile count and virtualization

Step 2: Accessibility

Even though this is a TV UI, add these accessibility features:

- `aria-label` on all focusable elements
- `role="grid"`, `role="row"`, `role="gridcell"` for the content grid
- `aria-selected` on the focused tile
- Live region for announcing focused content title to screen readers
- High contrast focus indicator (Netflix red border is good)

Netflix explicitly values accessibility. Mentioning this in an interview differentiates you.

Step 3: README

Write a README that covers:

- What this project demonstrates (Netflix TVUI engineering challenges)
- Architecture decisions and why (custom focus engine, virtualization, styling system)
- Performance metrics and how they're measured
- How to run locally
- Screenshots and/or GIF of the app in action
- Technologies used and NOT used (and why — e.g., "No CSS frameworks because TV apps don't use them")

Step 4: Deploy

```
bash
```

```
npm run build
```

```
# Deploy dist/ to Vercel, Netlify, or GitHub Pages
```

Step 5: Record a Demo

Record a 60-second video showing:

1. Keyboard navigation through the grid
2. Smooth scrolling when reaching row edges
3. Hero banner parallax
4. Detail overlay opening and closing
5. Performance HUD showing 60fps and low component count
6. Explain in voiceover: "This demonstrates the core challenges of Netflix TVUI engineering"

This video goes in your Netflix application AND your Twitter content.

Architecture Decision Records

Include these in your README or a separate DECISIONS.md:

ADR 1: Why no CSS framework?

Netflix TVUI doesn't use CSS. Gibbon renders to a custom surface with JavaScript-defined styles. By building our own styling system, we demonstrate understanding of why this matters: CSS cascade and specificity create unpredictable performance on constrained devices. Explicit, JS-object styles are predictable and GPU-friendly.

ADR 2: Why custom focus management instead of browser focus?

Browser focus (`tabIndex`, `:focus`) is designed for mouse+keyboard web apps. TV UIs need 2D spatial navigation (up/down/left/right), focus memory across rows, and debounced input handling. A custom focus engine gives us full control over these behaviors.

ADR 3: Why virtualization?

A Netflix home screen has 30+ rows with 15+ tiles each = 450+ items. On a device with 512MB RAM, mounting 450 React components simultaneously causes visible jank and potential OOM. Virtualization reduces mounted components to ~80 regardless of total content, keeping memory flat and renders fast.

ADR 4: Why requestAnimationFrame instead of CSS transitions for scrolling?

CSS transitions work great for simple state changes but become difficult to coordinate with

virtualization (new elements mounting during a transition). rAF-driven animations give us frame-by-frame control to synchronize scroll position with the virtual list's mount/unmount cycle.

ADR 5: Why Redux for a portfolio project?

Netflix TVUI uses Redux for state management. While simpler tools exist, using Redux demonstrates familiarity with their actual stack and proves you can manage complex state (focus position, scroll positions, UI states, content data) in a predictable, debuggable way.

Getting Started

```
bash
npm create vite@latest tv-ui -- --template react-ts
cd tv-ui
npm install @reduxjs/toolkit react-redux
npm run dev
```

Then tell Claude Code: "Read this plan and start with Phase 1 Step 2 — build the Focus Engine."

What This Proves to Netflix

1. **You understand TV UI engineering** — not just web development on a big screen
2. **You can build without frameworks** — custom styling, focus management, scrolling
3. **You think about performance first** — virtualization, metrics, budgets
4. **You know React deeply** — beyond ReactDOM, you understand the component model
5. **You write clean TypeScript** — strict types, proper architecture, documented decisions
6. **You can ship independently** — end-to-end from architecture to deployment
7. **You understand accessibility** — even on non-standard platforms

This is the project that gets you past the Netflix TVUI phone screen and into the onsite.